Grupo de Inteligencia Artificial
Universidad Rey Juan Carlos

Hoja de Problemas Tema 2
Búsqueda no-informada

Universidad Rey Juan Carlos

Ejercicio 1:

1.1. El enfoque de los Agentes Inteligentes concibe el objetivo de la Inteligencia Artificial como el intento de construir sistemas

(a) que "actúen" como las personas.

(b) que "actúen" de forma eficiente (racional) en su entorno.

(c) que "piensen" como las personas (tipo General Problem Solver, GPS).

(d) Ninguna de las respuestas es correcta.

1.2. ¿Para cual(es) de las siguientes tareas pueden construirse agentes basados en algoritmos de búsqueda en espacios de estados (p.e. la búsqueda de coste uniforme)?

(a) Encontrar una solución al problema de las Torres de Hanoi.

(b) Gestionar el tráfico rodado en una red de autopistas urbanas.

(c) Conducir un taxi en las calles de Barcelona.

(d) Jugar a las cuatro en raya contra un jugador humano.

1.3. ¿Cuáles de las siguientes afirmaciones acerca de los algoritmos de búsqueda no informados es (son) cierta(s)?

(a) Los algoritmos de búsqueda no informados requieren de información heurística para que sean óptimos.

(b) La búsqueda en amplitud es óptima y completa siempre y cuando el coste de los operadores sea constante.

(c) La búsqueda en profundidad es óptima y completa siempre que el coste de los operadores sea constante.

(d) Tanto la complejidad en tiempo como la complejidad en espacio de la búsqueda en amplitud se pueden expresar en función del número de nodos expandidos

1.4. Contemple el problema de búsqueda de la figura 1. ¿Cuál es el factor de ramificación del árbol de búsqueda generado por los métodos de búsqueda en el espacio de estados, si no se filtran estados repetidos?

(a) 0.5

(b) 2

(c) 4

(d) 6

1.5. Contemple el problema de búsqueda de la figura 1. En el árbol generado por la búsqueda en amplitud, si no se filtran estados repetidos, ¿cuántos nodos hay a nivel de profundidad k (suponiendo que la raíz tiene nivel 0, es decir $k \geq 0$)

(a) $k/4$

(b) 4^k

(c) $4*k^2$

(d) k^4

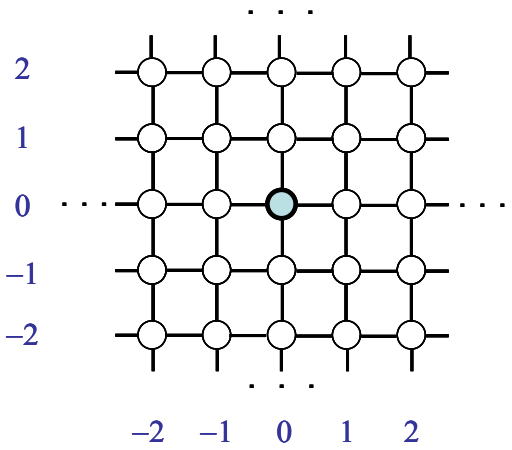


Figura 1. *Red 2D regular e infinito.*

- *Estado inicial (0,0).*
- *Estado meta (x,y); x,y ∈Z.*
- *Movimiento entre estados directamente conectados a coste 1*

Ejercicio 2:

En una mesa se encuentran dos jarras, una con una capacidad de 3 litros (llamada *Tres*), y la otra con una capacidad de 4 litros (llamada *Cuatro*). Inicialmente, *Tres* y *Cuatro* están vacías. Cualquiera de ellas puede llenarse con el agua de un grifo *G*. Asimismo, el contenido tanto de *Tres* como de *Cuatro* puede vaciarse en una pila *P*. Es posible echar todo el agua de una jarra a la otra. No se dispone de dispositivos de medición adicionales. Se trata de encontrar una secuencia de operadores que deje exactamente dos litros de agua en *Cuatro*.

a) Modele este problema como un problema de búsqueda. Con tal fin, defina el estado inicial, el conjunto de estados meta, los operadores (especificando sus precondiciones y postcondiciones), así como el coste de cada operador.

b) Caracterice el conocimiento a priori del agente de resolución del problema correspondiente? Facilite ejemplos de los resultados de la función expandir.

c) Encuentre una solución al problema.

Solución a)

Información relevante: sólo la cantidad de litros en cada jarra
Se modela un estado como un par (x, y), significando que hay x litros en *Cuatro* e y litros en *Tres*

- S = {0, 1, 2, 3, 4} x {0, 1, 2, 3}
- $s_0 = (0, 0)$
- s_m = {(2,0), (2,1), (2,2), (2,3)} ó: $meta?((x,y)) = \begin{cases} true\ si\ x = 2 \\ false\ de\ lo\ contrario \end{cases}$

Operadores:

- op_1: Llenar *Cuatro*
  - $(x, y) \rightarrow (4, y)$
  - Prec: $x < 4$
- op_2: Llenar *Tres*
  - $(x, y) \rightarrow (x, 3)$
  - Prec: $y < 3$
- op_3: Vaciar *Cuatro*
  - $(x, y) \rightarrow (0, y)$
  - Prec: $x > 0$
- op_4: Vaciar *Tres*
  - $(x, y) \rightarrow (x, 0)$
  - Prec: $y > 0$
- op_5: Llenar *Cuatro* desde *Tres*
  - $(x, y) \rightarrow (4, y - (4 - x))$
  - Prec: $x + y \geq 4, x < 4, [y > 0]$
- op_6: Llenar *Tres* desde *Cuatro*
  - $(x, y) \rightarrow (x - (3 - y), 3)$
  - Prec: $x + y \geq 3, y < 3, [x > 0]$
- op_7: Vaciar *Tres* en *Cuatro*
  - $(x, y) \rightarrow (x + y, 0)$
  - Prec: $x + y < 4, y > 0$
- op_8: Vaciar *Cuatro* en *Tres*
  - $(x, y) \rightarrow (0, x + y)$
  - Prec: $x + y < 3, x > 0$

Coste(op_i) = 1

Solución b)

Ejemplos función *expandir*:

- *expandir*((0,0)) = {(4,0) [op1] , (0,3) [op2]}
- *expandir*((4,0)) = {(0,0) [op3], (4,3) [op2], (1,3) [op6]}
- *expandir*((1,3)) = {(4,3) [op1], (0,3) [op3], (1,0) [op4], (4,0) [op5]}

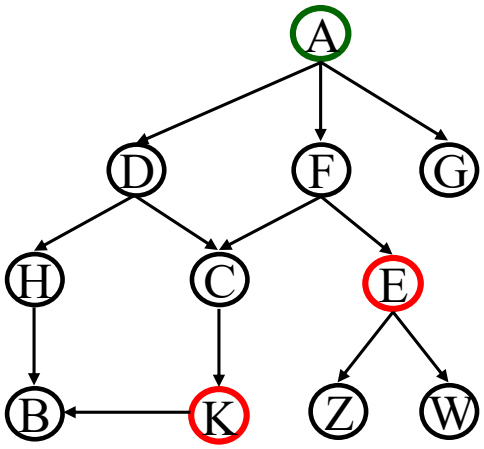
Los operadores están *implícitos* en la función *expandir* (de hecho, se pueden usar para "computarla", en vez de almacenar *expandir* en la memoria en forma de tabla).

Solución c)

P.e.: (0,0) –op_2–> (0,3) –op_7–> (3,0) –op_2–> (3,3) –op_5–> (4,2) –op_3–> (0,2) –op_7–> (2,0)

Ejercicio 3:

El grafo que se muestra al lado determina un problema de búsqueda. Cada nodo representa un estado; los arcos modelan la aplicación de operadores. Suponga que *A* es el estado inicial y que *K* y *E* son estados meta

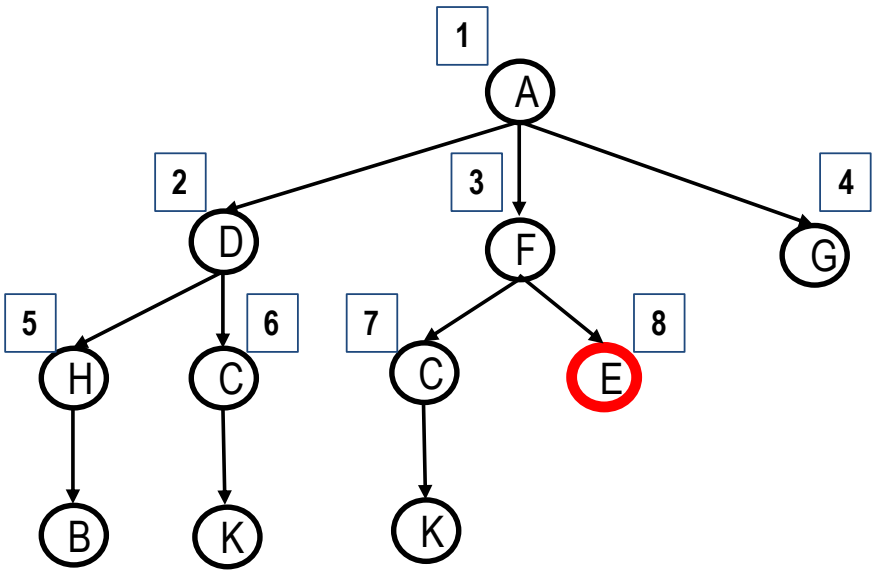


a) Desarrolle el árbol de búsqueda que genera la búsqueda en amplitud, *sin* filtrar estados repetidos. ¿Cuál de los nodos meta se encuentra primero?

b) Indique el orden en que se expanden los nodos

c) Ponga el estado de la lista abierta en cada paso del algoritmo

Solución 3a y b)



Solución 3c)

1. A
2. D F G
3. F G H C
4. G H C C E
5. H C C E
6. C C E B
7. C E B K
8. E B K K

Ejercicio 4:

a) ¿Cómo se podría instanciar el algoritmo de búsqueda genérico (de la diapositiva 20) para implementar una búsqueda en profundidad?

b) Para el grafo del ejercicio 3, desarrolle el árbol de búsqueda que genera la búsqueda en profundidad. Indique el orden en que se expanden los nodos ¿Cuál de los nodos meta se encuentra primero?

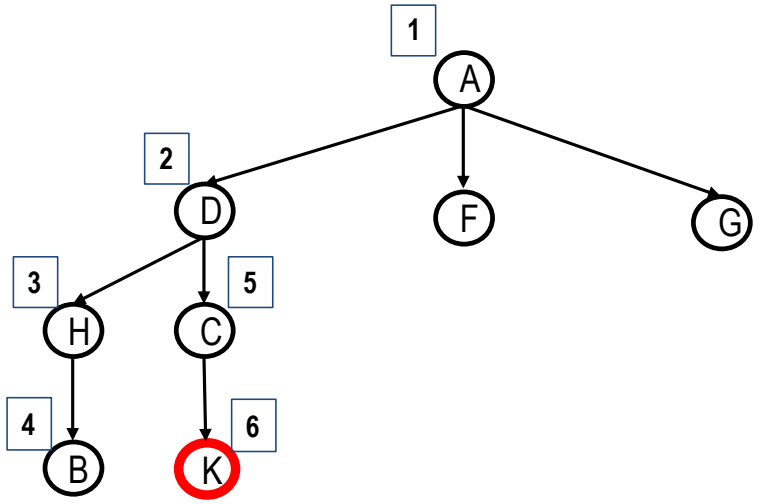
c) Haga un análisis de complejidad de dicho algoritmo, similar al de la diapositiva 28, asumiendo un límite de profundidad d^* fijado a priori.

Solución 4a):

Para implementar una búsqueda en profundidad, entre las hojas del árbol de búsqueda hay que elegir aquellos de mayor profundidad. Esto es equivalente a preferir las hojas que menos tiempo llevan en la lista *abierta*. Es decir, *abierta* ha de funcionar como una estructura LIFO. Para que *abierta* funcione como una pila (con comportamiento LIFO), y puesto que la función *primero* quita un elemento de la cabeza de la lista, también habrá que insertar nuevos elementos en la cabeza, e.d. la última línea del algoritmo sería:

```
ordInsertar(n,abierta,cabeza)
```

Solución 4b):



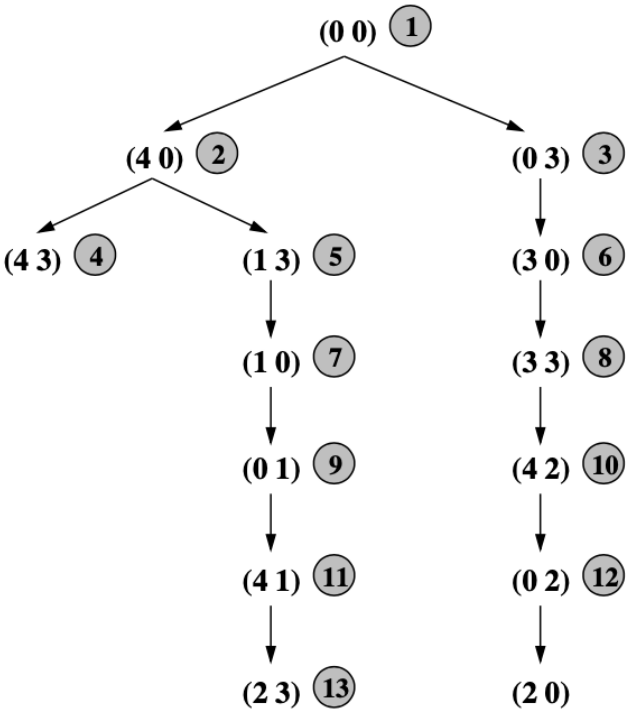
Solución 4c):

En general, la búsqueda en profundidad no es óptima (como en el ejemplo), y tampoco completa (salvo que haya un límite de profundidad d^* a priori, en el peor caso búsqueda en profundidad se puede perder en ramas infinitas). Respecto a la complejidad en tiempo, en el mejor caso es O(d) (se expande directamente el camino que lleva al mejor nodo meta de profundidad d), y en el peor caso O(b^{d^*}) (igual que la búsqueda en amplitud). Respecto a la complejidad en espacio, sobre la base de nuestra implementación (solución 5a), es igual que la de la búsqueda en amplitud. Sin embargo, una implementación más eficiente (p.e. recursiva) tendría una complejidad en espacio de O(b*d) (se almacenan los nodos de un "camino actual" y sus hermanos). Esto se debe a que, una vez que se expanda un nodo sin hijos (como el nodo *B* que se expande en el paso 4), es posible abandonar el camino actual, y liberar la memoria correspondiente (en el ejemplo los nodos *B* y *H*).

Ejercicio 5:

Considere el problema de las jarras del ejercicio 2. Simule la estrategia de búsqueda en amplitud para este problema, asumiendo que se filtran *todos los estados repetidos*. Indique el orden en que se expanden los nodos, y ponga el estado de la lista abierta en cada paso del algoritmo.

Solución 5:



Lista abierta:

1. **(0,0)**
2. **(4,0), (0,3)**
3. (0,3), **(4,3), (1,3)**
4. (4,3), (1,3), **(3,0)**
5. (1,3), (3,0)
6. (3,0), **(1,0)**
7. (1,0), **(3,3)**
8. (3,3), **(0,1)**
9. (0,1), **(4,2)**
10. (4,2), **(4,1)**
11. (4,1), **(0,2)**
12. (0,2), **(2,3)**
13. (2,3), **(0,2)**

Ejercicio 6:

Considere el siguiente problema:

> *Un hombre se encuentra en la orilla izquierda de un río junto con un lobo, una oveja y una col. Quiere cruzar el río llevando consigo el lobo, la oveja y la col. En la barca sólo hay dos plazas, una de las cuales debe ir ocupada por el hombre. Cada uno de los restantes pasajeros (lobo, oveja, col) ocupa una plaza, de tal modo que sólo uno puede acompañar al hombre en cada viaje. Además, no puede dejar solos en una orilla al lobo con la oveja, ni a la oveja con la col (ni los tres), porque la primera se comería a la segunda en cada paso*

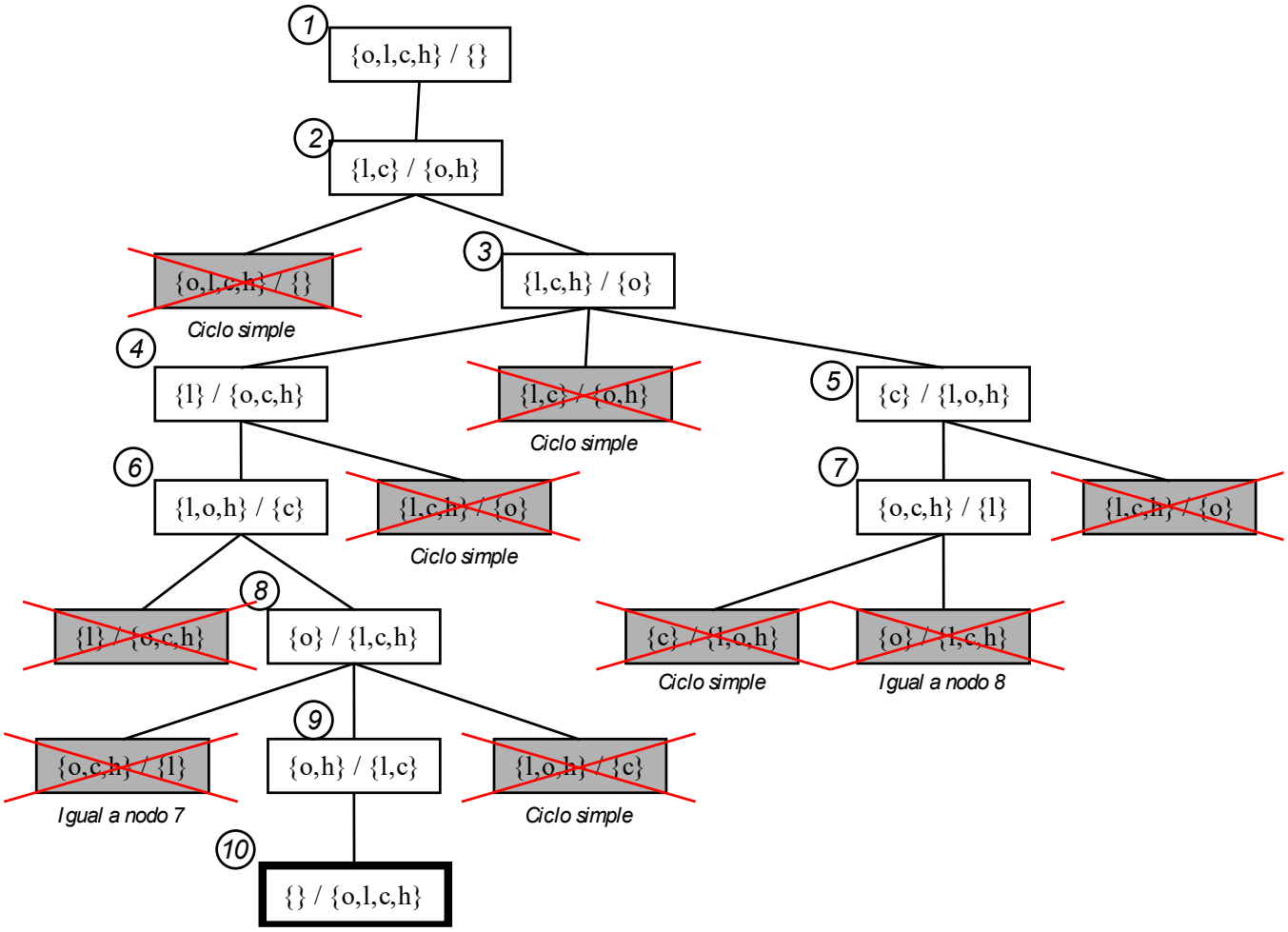
Suponga que se modela el problema como un espacio de estados, donde cada estado se describe como un par de conjuntos, indicando quien(es) se encuentran en cada orilla del río (c = col; o = oveja; l = lobo; h = hombre). Por tanto, el estado inicial del problema sería $(\{c,o,l,h\},\{\})$, y el estado meta $(\{\},\{c,o,l,h\})$.

a) Simule la estrategia de *búsqueda en amplitud* para este problema, asumiendo que se filtran *todos* los estados repetidos. Expanda los sucesores de los nodos siguiendo las preferencias del hombre, las cuales se ordenan (de mayor a menor) como sigue: viajar con la oveja, viajar con la col, viajar sólo, viajar con el lobo. Dibuje el árbol de búsqueda correspondiente e indique el orden en el que se exploran los nodos.

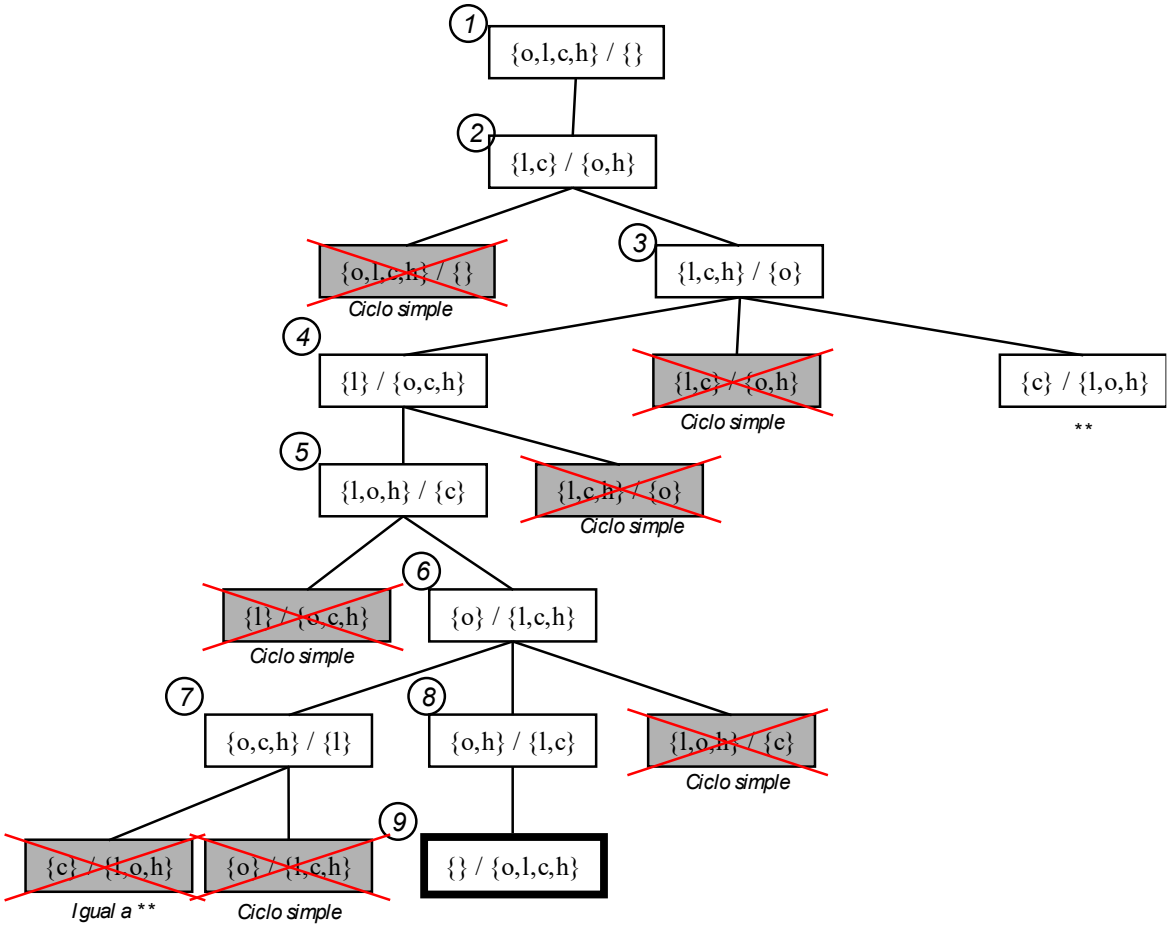
b) Simule ahora la estrategia de *búsqueda en profundidad* para este problema. Dibuje el árbol de búsqueda correspondiente e indique el orden en el que se exploran los nodos, asumiendo que se filtran *todos* los estados repetidos y siguiendo las preferencias del apartado anterior.

c) Suponga ahora que no se filtra ningún estado repetido. ¿La búsqueda en amplitud y la búsqueda en profundidad seguirían siendo completas? Razone brevemente su respuesta.

Solución 6a



Solución 6b



Solución 6c

Si se dejaran de filtrar estados repetidos, en principio se podrían dar secuencias infinitas de acciones. Por tanto, la búsqueda en profundidad ya no sería completa. En el ejemplo del apartado b), el proceso de búsqueda se metería en la rama infinita {o,l,c,h} / {} → {l,c} / {o,h} → {o,l,c,h} / {} → ...
Por otra parte, la búsqueda en amplitud del apartado a) seguiría siendo completa. De hecho, siempre es completa (si el coste de los operadores es constante), aun cuando el conjunto de estados posibles es infinito.

Ejercicio 7:

Suponga que en la red de carreteras de Rumania presentada en clase (diapositiva 31) nuestra agente se encuentra en Caiva (C) y desea trasladarse a Fagaras (F)

a) Desarrolle el árbol de búsqueda que genera la *búsqueda de coste uniforme*, indicando los valores de *g* para cada nodo. Suponga que se filtran *ciclos simples*.

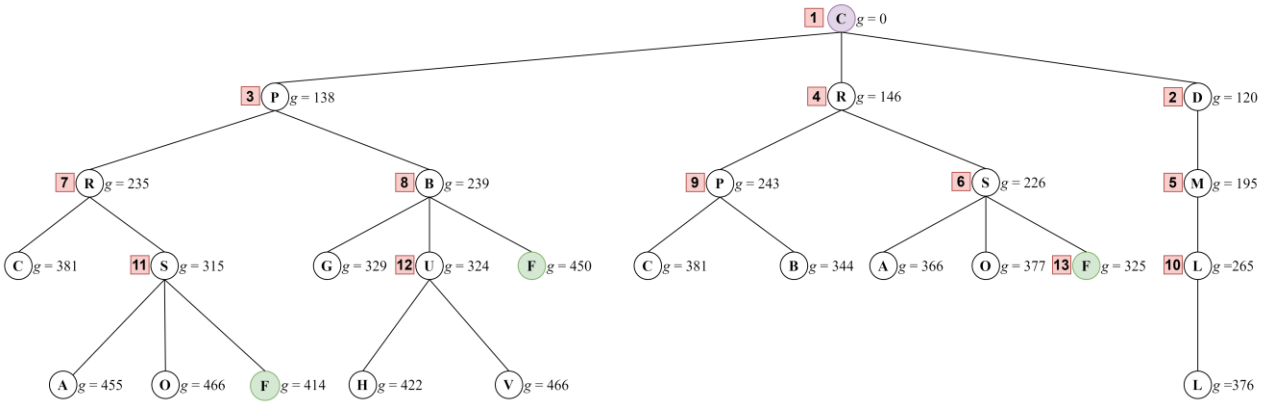
b) Indique el orden en que se expanden los nodos

c) Ponga el estado de la lista abierta en cada paso del algoritmo

Solución 6:

En lo que sigue, se supone que se filtran ciclos simples.

a y b):



c)

1. (C,0)
2. (D,120) (P,138) (R,146)
3. (P,138) (R,146) (M,195)
4. (R,146) (M,195) (R,235) (B,239)
5. (M,195) (S,226) (R,235) (B,239) (P,243)
6. (S,226) (R,235) (B,239) (P,243) (L,265)
7. (R,235) (B,239) (P,243) (L,265) (F,325) (A,366) (O,377)
8. (B,239) (P,243) (L,265) (S,315) (F,325) (A,366) (O,377) (C,381)
9. (P,243) (L,265) (S,315) (U, 324) (F,325) (G,329) (A,366) (O,377) (C,381) (F,450)
10. (L,265) (S,315) (U, 324) (F,325) (G,329) (B,344) (A,366) (O,377) (C,381) (C,381) (F,450)
11. (S,315) (U, 324) (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,450)
12. (U,324) (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,414) (F,450) (A,455) (O,466)
13. (F,325) (G,329) (B,344) (A,366) (T,376) (O,377) (C,381) (C,381) (F,414) (H,422) (F,450) (A,455) (O,466) (V,466)